Rakamin engineering assignment

Missing ratings in interview portfolios

A Rails and React change to preserve unassessed skills through review, role comparison and export.

Repository	Working copy
rakamindev/ai-interview-platform	C:/Rakamin2
Baseline	main at b836d02
Feature branch	feat/evidence-first-assessment
Work dates	29-30 October 2026

Problem addressed

A missing model rating could become level 1 through numeric coercion. For example, an interview that never assessed SQL could still produce a low SQL score. The revised pipeline stores that skill as not assessed and keeps the same meaning in the assessor screen and exports.

Local verification

47 backend examples, 11 component tests and 3 browser workflows passed. TypeScript and the production build passed. A deliberate missing-rating fault failed the existing regression test; reverting it restored the test.

Report contents

Repository and local setup 02
Users and assessment policy 03
Findings and priorities 04
Acceptance criteria 05
Implementation decisions 06
Tests and corrections 07
Desktop review 08
Mobile and failed saves 09
Delivery and release notes 10

Prepared with AI assistance. Demonstration records are synthetic. The source, command logs and technical-defense notes accompany this report.

Step 1 / Setup

01

Repository and local setup

The project runs from C:/Rakamin2. The original main branch remains at b836d02; implementation commits are on feat/evidence-first-assessment. The code review followed assessment setup, interview sessions, portfolio generation, overrides, vacancy comparison and export.

Layer	Environment and work completed
API	Ruby 3.3.2, Rails 7.0 and PostgreSQL. Added RSpec coverage for validation, persistence, requests and background work.
Web	React 18, TypeScript, Vite, Tailwind and Radix. Added Vitest, Testing Library, Playwright and a dependency lockfile.
Local database	A dedicated PostgreSQL cluster listens on localhost:54329. Tests check persisted rows, null ratings, rollback and concurrent jobs.
Model response	A deterministic injected response exercises the real validator and persistence code without calling Gemini.
Windows tooling	Task-local Ruby and database tools are in the ignored .runtime directory. Dependencies and local credentials are excluded from the source archive.

Commands used

API

```
bundle exec rails db:prepare
bundle exec rspec --format documentation
```

Web

```
npm ci
npm test
npm run build
npm run test:e2e
```

What the local run covers

Verification starts with a completed interview and covers portfolio generation, human review, comparison and export. The recording uses the running Rails API and synthetic records. No live Gemini or microphone interview was run. Docker was unavailable; compose configuration is included as an untested alternative.

The brief gives a deadline of Wednesday 19 August, 13:00 WIB. This work was prepared on 29-30 October 2026. Submission availability needs confirmation with the recruiter.

Step 2 / Product context

02

Users and assessment policy

The product specification calls for probing capability and using transcript evidence. Its workflow connects the interview portfolio to reviewer overrides and vacancy comparison. [1, 2] A rating error can therefore reach several screens and exported files.

The user needs below are assumptions drawn from that workflow and the assignment brief. No recruiter or candidate interviews were conducted.

User	Need reflected in the change
Assessor	Read the supporting excerpt in context before accepting or changing a rating. Require a reason for an override.
Recruiter	See which skills need another assessment before sharing the portfolio. Keep those gaps visible in exports.
Hiring manager	Compare evidence with actual role requirements. Distinguish a demonstrated skill gap from a requirement the interview missed.
Candidate	Avoid a low score for a skill that was never assessed. Keep a human reviewer responsible for interpreting the evidence.

Handling candidate data in Indonesia

UU 27/2022 includes correction and access rights in Articles 6-7, objections to certain solely automated decisions in Article 10, and impact assessment for high-risk processing in Article 34. It also addresses purpose, accuracy and confidentiality. [3]

The implementation binds evidence access to the authenticated tenant, filters sensitive log fields and avoids exposing raw provider errors. Reviewers can record a reason for changing an assessed rating. These are engineering safeguards; the work does not establish legal compliance.

Policy questions before production

The controller still needs to decide legal basis, retention, deletion, candidate correction and objection processes, provider arrangements and any required impact assessment. Language effects and fairness need evaluation with consented data.

Step 3 / Code review

03

Findings and priorities

Priority / type	Observed behavior	Change or remaining decision
P0 / defect	Portfolio and override lookups were not joined through the authenticated tenant.	Tenant-scoped lookups and foreign-ID request tests.
P0 / policy	Password login could select an organization without a user-membership relation.	Restrict local login to opt-in development use. Production identity remains an integration requirement.
P1 / defect	Numeric coercion and clamping converted missing or malformed ratings to valid-looking levels.	Validate types; store invalid or missing ratings as not assessed.
P1 / defect	Generation deleted old skill rows before validating all replacements.	Validate first, then replace rows and completion status in one transaction.
P1 / defect	Duplicate or exhausted jobs could regenerate or mark completed work as failed.	Serialize by session, skip completed work and guard exhaustion updates.
P1 / undefined	Quote provenance, unassessed states and confidence limits had no clear rules.	Define acceptance criteria and implement a response validator.
P1 / inference	The comparison prompt requested culture fit and hiring advice from score patterns.	Use deterministic skill comparison; remove those inferences.
P2 / contract	The API sent expected_level while the UI read required_level.	Use the API field consistently and test the contract.
P2 / interface	Request errors were swallowed. Long text and enlarged text broke parts of the mobile layout.	Add retry feedback and fix the layouts caught by browser tests.
P3 / setup	Setup examples used inconsistent API ports and frontend folder names.	Correct the defaults and add a runbook.

P0 covers candidate-data access; P1 covers misleading results or evidence loss; P2 covers interpretation and recovery; P3 covers setup. These priorities reflect expected impact, not measured incident rates.

Step 4 / Definition of done

04

Acceptance criteria

These criteria were committed before implementation. The full list and trade-offs are in assessment/acceptance-criteria.md.

Boundary	Required behavior
Rating type	Accept only integer levels 1-5. Reject null, strings, floats, booleans and out-of-range values as ratings.
Coverage	Return every configured skill. Omitted or untouched skills remain explicitly unassessed.
Evidence	Require 1-3 bounded excerpts found in candidate turns after whitespace normalization. Interviewer-only or invented quotes cannot support a rating.
Confidence	Accept known confidence values and cap them by interview coverage and probe count. Confidence describes the signal, not candidate ability.
Response shape	Unknown or duplicate identities and malformed JSON fail the attempt without replacing existing rows.
Publication	Commit all replacement rows and completion status together. Validation or database failure restores the previous rows.
Jobs	Serialize duplicate jobs and skip completed work. A queue failure stays retryable; an older exhausted job cannot overwrite completion.
Historical data	Mark old ratings as legacy without changing their levels or claiming verified provenance. Refuse rollback that would require inventing levels.
Override	Allow overrides only for assessed rows. Require a reason and retain the original AI level, reviewer and timestamp.
Authorization	Return 404 for foreign-tenant resources. Candidate roles cannot read assessor evidence. Background comparison checks both tenants.
Role comparison	Use expected_level. Different canonical IDs cannot match by label; ambiguous custom matches remain unassessed.
Interface	Cover loading, empty, error, partial, legacy and long-text states, keyboard focus, 390px layout and 200% text resizing.

Step 4 / Approach

05

Implementation decisions

Option	Effect	Trade-off
UI warning only	Helps a reviewer notice missing evidence.	Leaves coerced scores and destructive replacement behavior in the API and export. Rejected.
Validation and atomic writes	Chosen. Checks evidence before publishing a complete set of skill rows.	Requires a schema change and holds a database connection and session lock during the bounded model call.
Versioned ledger and outbox	Adds review history and more durable job scheduling.	Needs more migrations and operational work. Kept as a follow-up to the immediate rating fix.

Data path

Interview	Validation	Publication	Review	Comparison
Ended session and transcript	Types, coverage and candidate quotes	Skill rows and status in one transaction	Evidence and attributed override	Role requirements and export

Schema and rollback

The migration adds assessment_status and assessment_reason, and makes ai_level nullable. A database check requires a null level for not_assessed and a level for assessed or legacy. Existing scores retain their values and receive legacy status.

Tests round-trip legacy rows through the migration. The down migration refuses to run while null ratings exist. An application rollback can retain the additive schema; a schema rollback first needs a decision about those unassessed records.

Timeouts and retries

The portfolio call has a 180-second HTTP read timeout and no nested provider retries. Sidekiq handles durable retries. The timeout is not an end-to-end deadline, and the session lock does not guarantee exactly-once provider billing.

Quote matching is deliberately conservative: a paraphrase can be rejected. It checks where a quote came from; a reviewer must still judge whether the quote supports the rating.

Step 5 / Verification

06

Tests and corrections

Check	Recorded result
Backend	47 examples passed against PostgreSQL, covering validation, tenant requests, rollback, jobs and concurrent duplicates.
Frontend	11 component tests and 3 browser workflows passed. TypeScript and the Vite production build passed.
Real API	Four skills returned: two assessed, two null. SQL stayed unassessed with expected_level=4. JSON and PDF exports passed.
Browser controls	9 live checks passed for PDF downloads, navigation, empty-role action and logout, with no page errors.

Deliberate regression

On test/seeded-missing-rating, the missing-rating result was deliberately changed from null to level 1. The existing regression test failed. Reverting the change restored the test without changing its assertion.

```
Fault: fbd99f85174bc508ad4b18325176fd73b2ac9829
Revert: dc5058b2df147f72d351a5491d097e13633d1a31
```

1 example, 1 expected failure; after revert, 1 example, 0 failures. Assertions unchanged.

Corrections during verification

The first AI-generated savepoint implementation still saved an extra skill row after rollback. Rails retained the rolled-back association target and autosaved it when recording failure. Reloading the portfolio before that update cleared the cached target. The original failing output is in ai-verification-red.log; the assertion stayed unchanged.

TypeScript also caught a nonexistent regeneration method and an incorrect assumption that a union response always contained a portfolio. Browser checks caught squeezed skill headings on phones and navigation overflow at 200% text size. Those checks passed after correcting the code and layout.

These results cover synthetic fixtures and an injected model response. They do not validate live Gemini quality, audio interviews, model calibration or hiring fairness.

Step 5 / Screenshots

07

Desktop review

The review shows coverage before individual skill ratings. This capture uses a synthetic browser fixture with one assessed skill and one skill requiring follow-up.

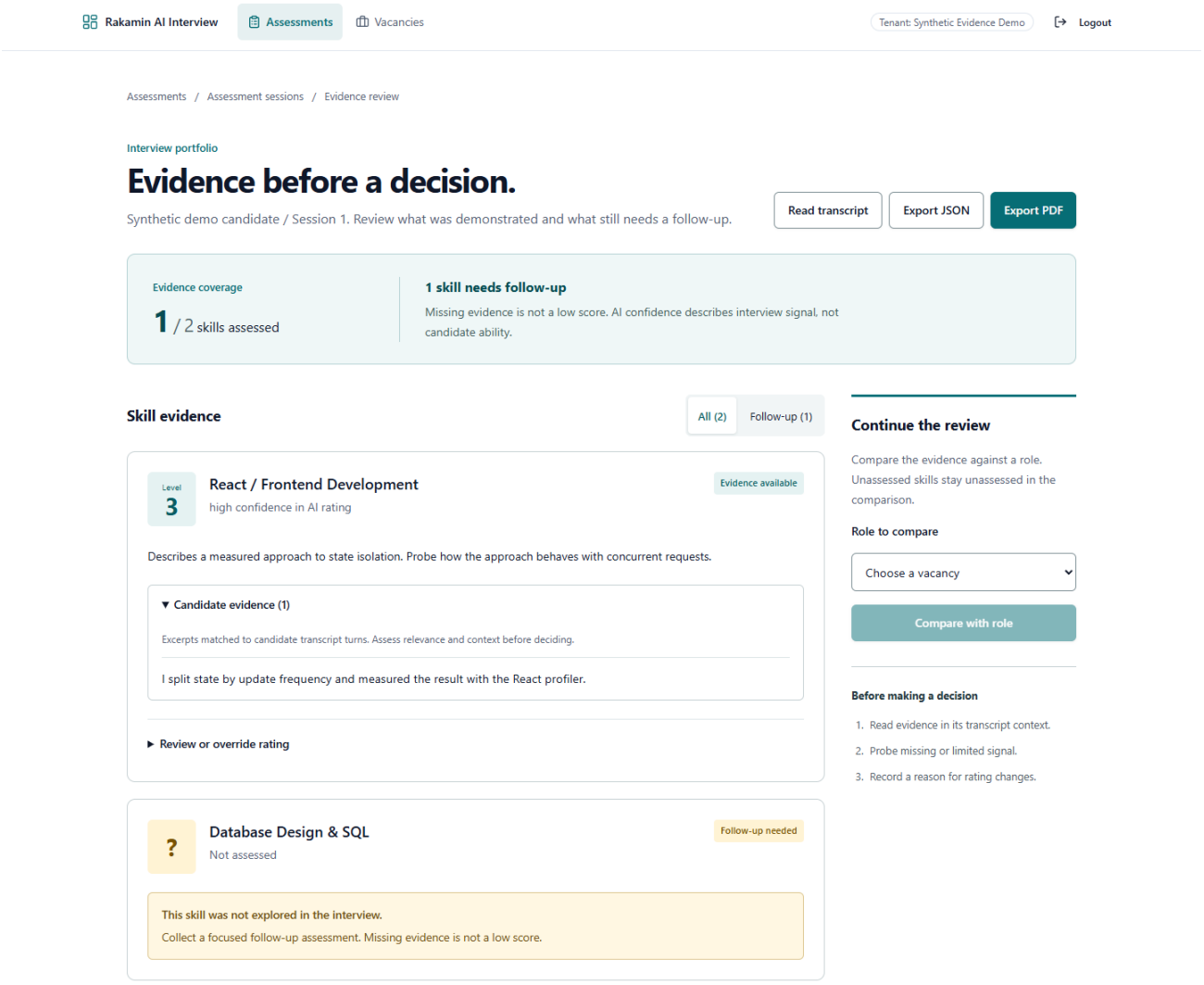


Figure 1. The fixture API supplies the counts and statuses. The SQL row has no score; its follow-up action remains visible.

Review order

Each assessed row puts the summary and candidate excerpt above the override form. Reviewers can open the transcript before changing a rating. The role selector sits alongside the evidence so a comparison is available from the same workspace.

The application retains the repository's teal and yellow styling. Native disclosure controls keep long excerpts manageable. The contrast check sampled 177 visible text and control combinations across review, failed save and comparison, with no AA text-threshold failures. Disabled controls were excluded.

Step 5 / Screenshots

08

Mobile and failed saves

Browser tests exercise the review at 390px, with long unbroken text and with text enlarged to 200%. They check for horizontal page overflow.

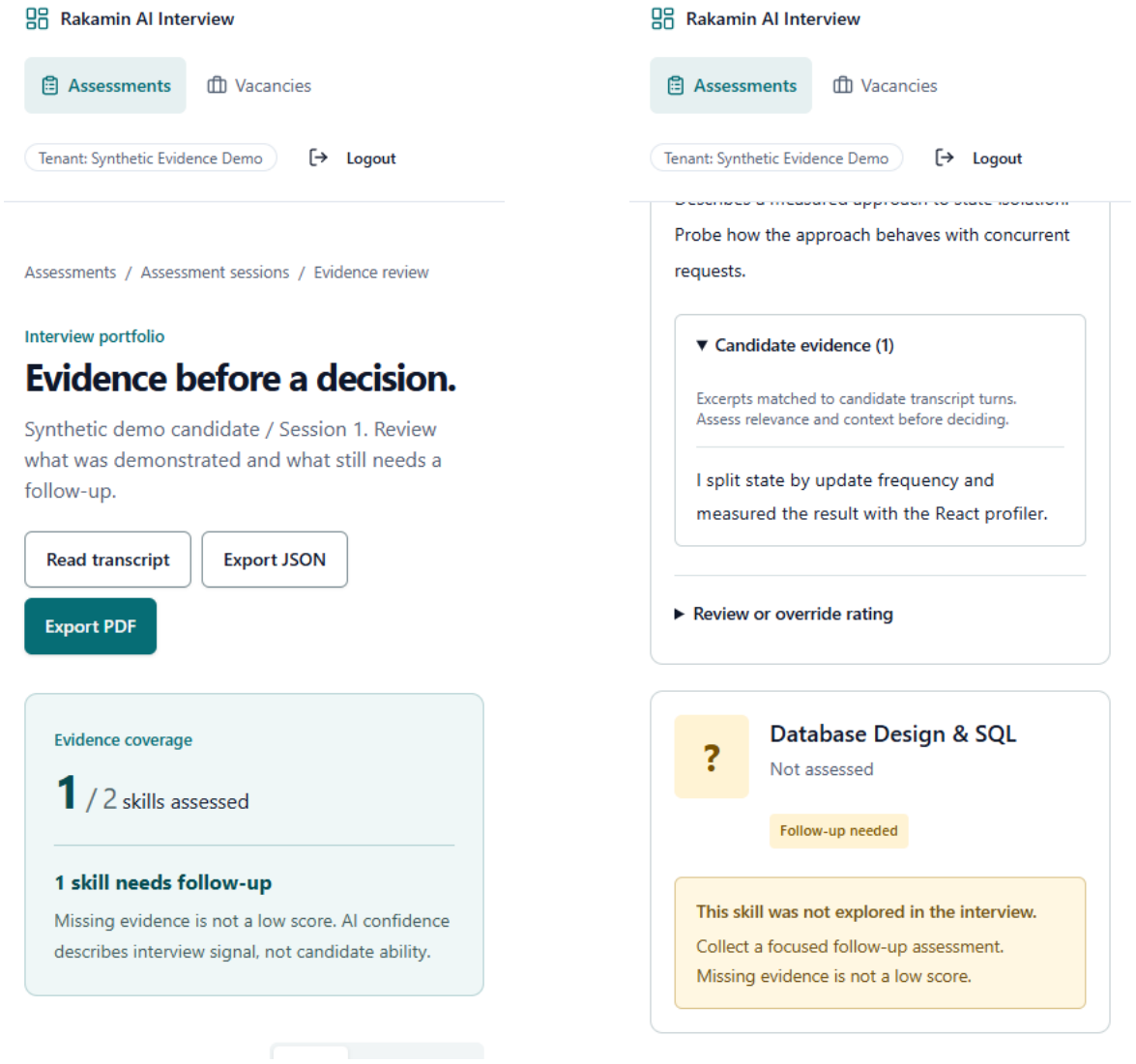


Figure 2. Phone overview and evidence detail. Full-resolution captures are included in assessment/screenshots/.

Failed override save

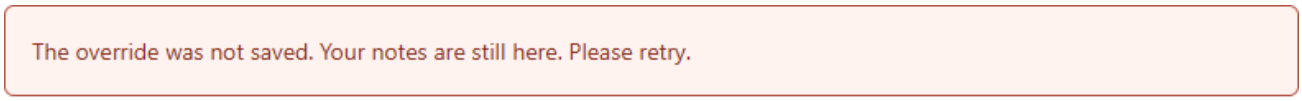


Figure 3. The test deliberately fails a save. The form keeps the reviewer's reason, reports that it was not saved and allows another attempt. The retry succeeds.

Separate browser checks cover request retry, empty results, pending generation and failed generation. The recording labels its simulated failures and then shows recovery through the real API.

Delivery

09

Delivery and release notes

Artifact	Contents
api/ and web/	Implementation and tests on feat/evidence-first-assessment.
assessment/	Command logs, verification.json, screenshots and sample exports.
walkthrough.webm	Captioned local recording, 220 seconds. No narration.
Supporting notes	RUNBOOK.md, technical-defense.md and a prepared PR_DESCRIPTION.md.
Portable package	Rakamin2-submission.zip includes source and artifacts. evidence-first.bundle preserves the project branches and fault/revert history.

Submission links

Pull request: not published; the local material is included in the package.

Walkthrough: not published; the local material is included in the package.

Before submitting, publish the prepared PR, host the recording and add both links here. No portal submission has been made.

Release sequence and remaining work

Apply the additive migration, then deploy the API and web together so both understand nullable ratings. Review legacy portfolios and arrange follow-up assessments. Monitor failed generation, pending-job age, provider latency, missing evidence and overrides.

The implementation keeps only the latest override. An append-only review ledger, durable outbox and abandoned-job watchdog remain follow-ups. Production also depends on trusted organization identity and the controller's privacy policies. PDF font coverage beyond Latin is unverified.

Sources

1. Product behavior specification

2. Product workflow simulation

3. UU 27/2022, JDIH Kemkomdigi

4. Repository baseline b836d02